Sequences — Types, Arithmetic and Geometric

A 1.5–2 hour lecture on sequence notation, explicit/recursive forms, arithmetic and geometric patterns and CS applications.

• Recommended teaching duration: approximately 1.5–2 hours including board work, worked examples, Python demonstrations and student discussion.

• Teaching approach: concept → notation → worked example → CS interpretation → Python verification → student practice.

Lecture Roadmap

• Concept introduction and terminology

• Detailed definitions and notation

• Worked mathematical examples

• CS case studies with complete solutions

• Python implementation and verification

• Extended question bank

• 2–3 unsolved student practice questions at the end

1. Definition and Notation

A sequence is an ordered list a1,a2,a3,... indexed by positive integers. Unlike a set, order and position matter.

2. Explicit and Recursive Forms

An explicit formula computes a_n directly. A recursive definition specifies initial term(s) and a rule for producing the next term. Recursive thinking is important in algorithms.

3. Arithmetic Sequences

A constant difference d gives a_n=a1+(n−1)d. Examples: 4,7,10,13,... and 20,17,14,11,...

4. Geometric Sequences

A constant ratio r gives a_n=a1r^(n−1). Examples: 2,6,18,54,... and 64,32,16,8,...

5. Finding Missing Terms

Use the constant difference or ratio to determine unknown terms. Always check whether the pattern is arithmetic or geometric before applying a formula.

6. CS Applications

Arithmetic growth can model fixed increments such as four additional servers per deployment. Geometric growth can model duplication, exponential search spaces and repeated replication.

7. Python Generation

List comprehensions and loops can generate sequence terms. Compare generated values with the mathematical formula.

8. Case Study — Cloud Workers

A service begins with 8 workers and adds 4 after each deployment.

Solution: arithmetic, a_n=8+4(n−1). The 10th value is 44.

9. Case Study — Replication

A backup starts with 2 copies and doubles at every stage.

Solution: geometric, a_n=2·2^(n−1). The 8th value is 256.

10. Case Study — Network Traffic

A monitoring system records 100 requests in minute 1 and increases by 25 requests per minute for a short test. This is arithmetic under the stated model.

Solution: a_n=100+25(n−1); a_12=375.

11. Recursive Programming Link

A recursive sequence such as a_n=a_(n−1)+d mirrors a simple recursive function. Students should distinguish mathematical recurrence from a loop implementation.

Python Laboratory

Python Activity 1

```
def arithmetic_sequence(a1,d,n):
    return [a1+(i-1)*d for i in range(1,n+1)]

def geometric_sequence(a1,r,n):
    return [a1*r**(i-1) for i in range(1,n+1)]

print(arithmetic_sequence(8,4,10))
print(geometric_sequence(2,2,8))
```

Ask students to predict the output before executing the code, then change at least one input and explain the new result.

Python Activity 2

```
# Recursive-style generation
def arithmetic_recursive(a,d,n):
    result=[a]
    for _ in range(n-1):
        result.append(result[-1]+d)
    return result

print(arithmetic_recursive(5,3,8))
```

Ask students to predict the output before executing the code, then change at least one input and explain the new result.

Extended Question Bank — Instructor-Led Practice

The following questions are designed to provide enough board/classroom practice for a 1.5–2 hour session. They cover the same topic areas as the relevant VU MTH202 sequence, but are written as fresh exercises rather than reproducing the handout wording.

1. Find the next three terms of an arithmetic sequence.
2. Find the common difference.
3. Find the nth term of an arithmetic sequence.
4. Find the common ratio of a geometric sequence.
5. Find the nth term of a geometric sequence.
6. Determine whether a given sequence is arithmetic, geometric or neither.
7. Find a missing term in an arithmetic sequence.
8. Find a missing term in a geometric sequence.
9. Model server capacity as a sequence.
10. Model data replication as a sequence.
11. Generate 20 terms in Python.
12. Compare arithmetic and geometric growth after 10 steps.
13. Explain why geometric growth can create resource problems in computing.

End-of-Lecture Student Practice — No Solutions

1. Model a CS workload as arithmetic or geometric and justify.
2. Find a requested term using the appropriate formula.
3. Generate the sequence in Python and compare.

Quick Recap

Sequences model ordered patterns; arithmetic and geometric forms capture two important growth types.